SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

ASSESSMENT TOOL 2

Scenario-Based Assignment (CO2)

Field	Details
Course Name	Cryptography and Network Security
Course Code	CSA51
Course Outcome	CO2: Analyze Public Key crypto system strategies using RSA and key exchange for encryption algorithms to strengthen information security (BL4, BL5)
Assessment Type	Scenario-Based (20%)
Total Marks	50
Student Name	Dudekula Mohammad Ilyas
Register Number	192325017

Annexure A – Code of Conduct

I, **Dudekula Mohammad Ilyas** (Reg No: **192325017**) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: ***Mohammad Ilyas***

Name: Dudekula Mohammad Ilyas

Reg No: 192325017

Date: 21 /10/26

Contents

1. **Question 3 – Key Exchange in an Unsecured Network**
2. **Question 4 – Public Key Cryptography for Email Security**

List of figures: Fig. 1 DH message exchange; Fig. 2 DH flowchart; Fig. 3 Man-in-the-Middle attack; Fig. 4 Authenticated DH (STS); Fig. 5 Validation flowchart; Fig. 6 RSA key generation; Fig. 7 RSA encryption/decryption; Fig. 8 Digital signature; Fig. 9 Hybrid secure e-mail; Fig. 10 PKI hierarchy; Fig. 11 Key life cycle.

Question 3: Key Exchange in an Unsecured Network

Scenario Understanding

Two users (Alice and Bob) want to set up a secure communication channel over a **public, untrusted network** (such as the Internet) and have chosen the **Diffie-Hellman (DH) key exchange**. Symmetric encryption such as AES is fast, but it needs both sides to hold the same secret key, and sending that key over an open network would expose it. DH solves this **key distribution problem**: it lets two parties who have never met derive an identical shared secret while an eavesdropper who sees every message cannot compute it. The scenario also states that the network is **vulnerable to attacks**, so the solution must go beyond the basic protocol and address authentication of the exchanged values.

Part A: How the Diffie-Hellman Key Exchange Works

A.1 Mathematical foundation

DH, published by Whitfield Diffie and Martin Hellman in 1976, was the first practical public-key scheme. Its security depends on the **Discrete Logarithm Problem (DLP)**: given a prime p, a generator g and a value y = g^x mod p, it is computationally infeasible to find x when p is large (2048 bits or more). The reverse operation, modular exponentiation, is easy. This one-way property is what makes the exchange safe.

- **p** – a large prime number (public).
- **g** – a primitive root (generator) of p, 1 < g < p (public).
- **a, b** – private keys chosen secretly and randomly by Alice and Bob, 1 < a, b < p-1.
- **A = g^a mod p, B = g^b mod p** – public values exchanged over the network.

A.2 Step-by-step protocol

Step	Alice	Public network	Bob
1	Agree on public parameters	p and g are sent/known to everyone	Agree on public parameters
2	Chooses secret a randomly	—	Chooses secret b randomly
3	Computes A = g^a mod p	—	Computes B = g^b mod p
4	Sends A to Bob	A --> and <-- B	Sends B to Alice
5	K = B^a mod p	—	K = A^b mod p
6	Both now hold the same secret K = g^(ab) mod p		

Figure 1 shows the sequence of messages exchanged between Alice and Bob over the public network, and **Figure 2** presents the same procedure as a flowchart, including the check that both sides arrive at the same key.

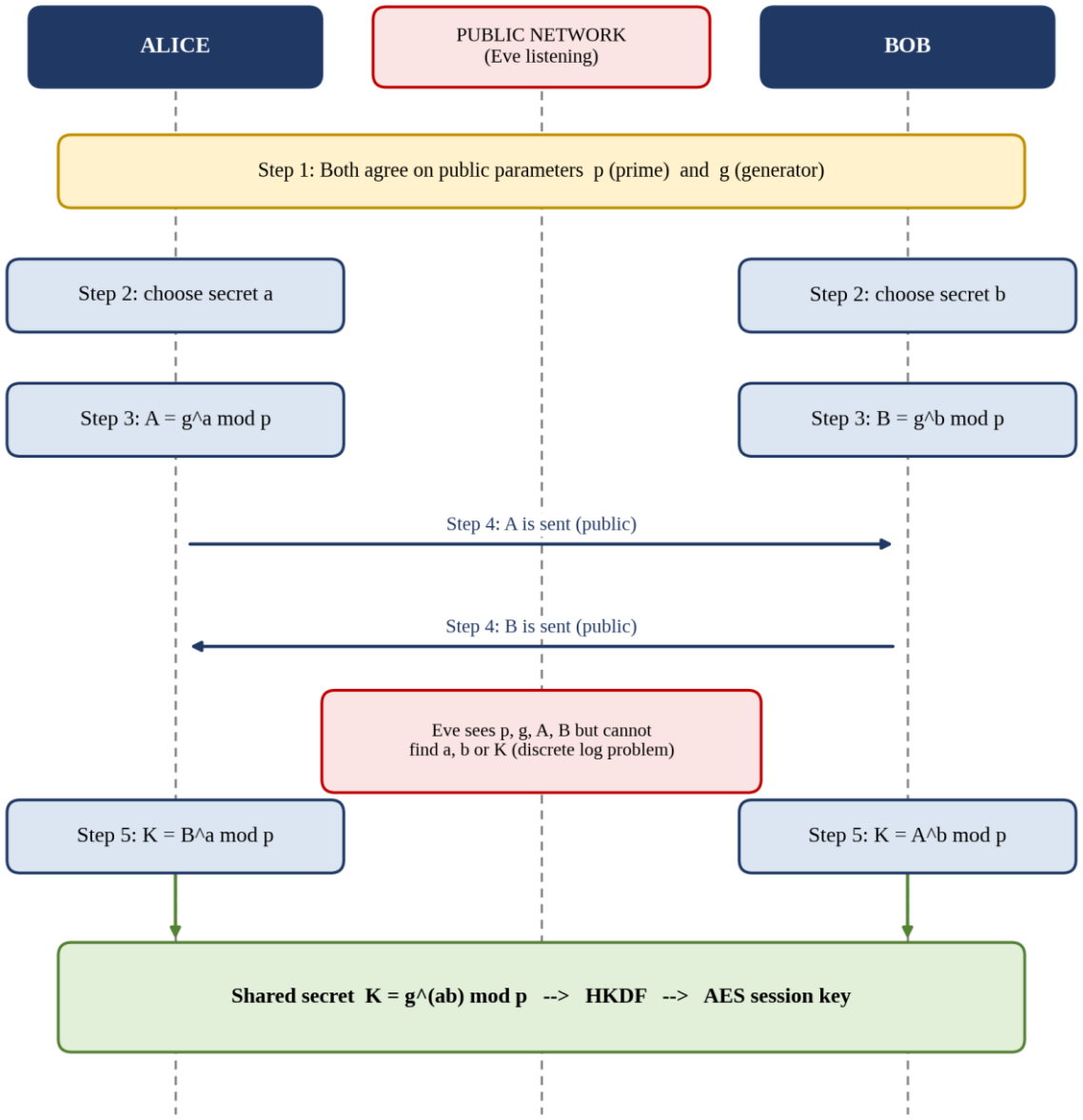


Figure 1: Diffie-Hellman key exchange – message sequence diagram

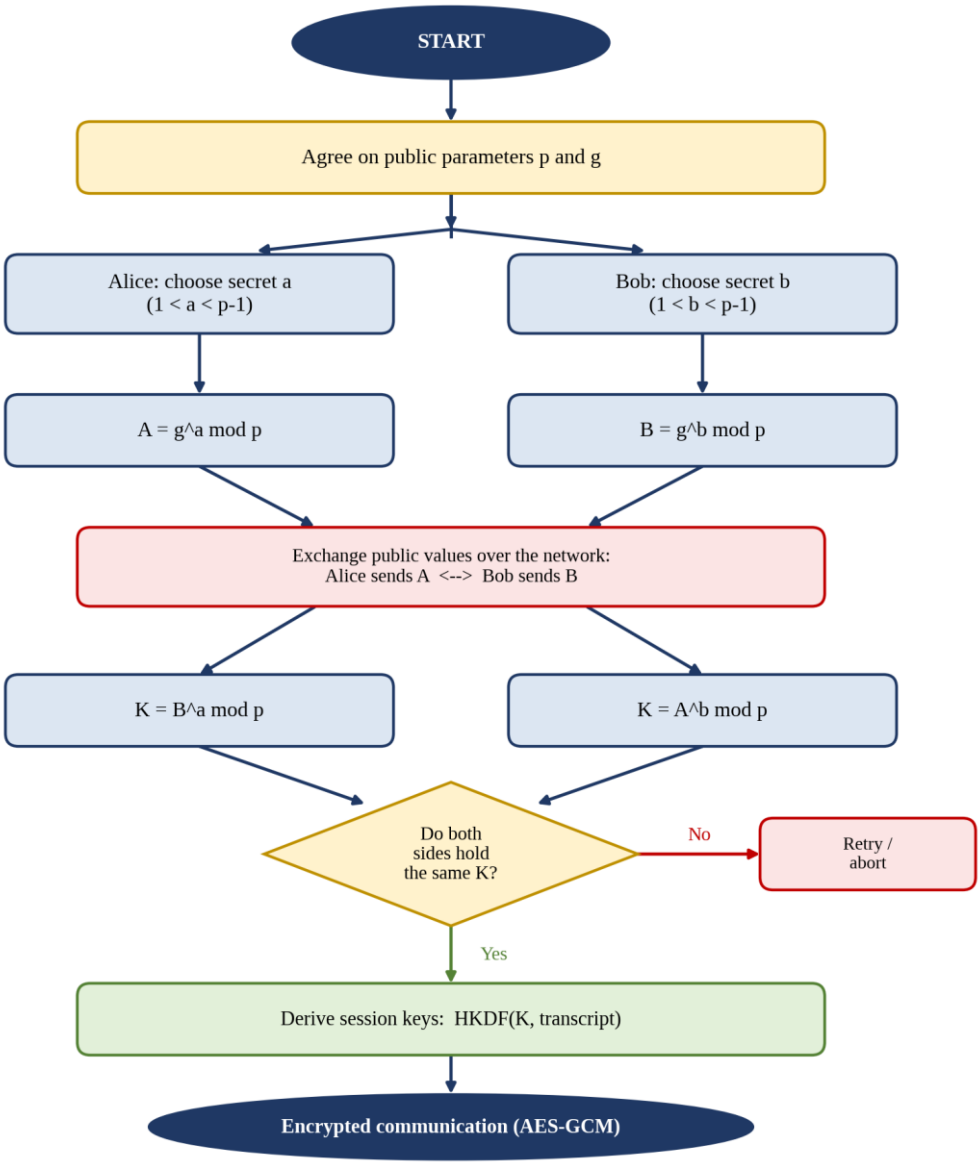


Figure 2: Flowchart of the Diffie-Hellman key exchange procedure

The two computations agree because exponentiation is commutative:

```
K(Alice) = B^a mod p = (g^b)^a mod p = g^(ab) mod p
K(Bob)   = A^b mod p = (g^a)^b mod p = g^(ab) mod p
```

A.3 Worked numerical example

Small numbers are used for clarity (real systems use 2048-bit or larger values). Let p = 23 and g = 5. Alice chooses a = 6 and Bob chooses b = 15.

Item	Alice	Bob
Private key	a = 6 (secret)	b = 15 (secret)
Public value	A = 5^6 mod 23 = 15625 mod 23 = 8	B = 5^15 mod 23 = 19
Exchanged	Sends 8 to Bob	Sends 19 to Alice
Shared secret	K = 19^6 mod 23 = 2	K = 8^15 mod 23 = 2

Both users obtain **K = 2** without ever transmitting it. An eavesdropper (Eve) knows p = 23, g = 5, A = 8 and B = 19 but not a or b. With such a small prime she could brute-force it, but with a 2048-bit prime the search space is astronomically large.

A.4 From shared secret to session key

The raw value K is not used directly as an encryption key. Both parties feed it, together with a transcript of the handshake, into a **Key Derivation Function (KDF)** such as HKDF to produce one or more session keys (for example, a 256-bit AES key and an HMAC key). Data is then protected with a fast symmetric cipher such as AES-GCM. DH therefore acts only as the secure "key-agreement" step, while bulk data is handled by symmetric cryptography.

A.5 Why an eavesdropper cannot recover the key

Eve sees p, g, A and B. To get K = g^(ab) she must either solve the DLP to find a or b, or solve the **Computational Diffie-Hellman (CDH) problem** (compute g^(ab) from g^a and g^b). No efficient classical algorithm is known for either when p is a strong 2048-bit prime. This gives the exchange passive security: a purely listening attacker learns nothing useful.

A.6 Advantages and limitations

Advantages	Limitations
No secret key has to be transmitted or pre-shared	Does **not authenticate** either party, so it is open to man-in-the-middle attacks
Each party contributes randomness to the key	Computationally heavier than symmetric operations (but used only once per session)
Supports forward secrecy when ephemeral keys are used (DHE/ECDHE)	Weak or reused parameters (small primes, bad generators) reduce security
Widely standardised (TLS, IPsec/IKE, SSH, Signal)	Basic DH is vulnerable to quantum computers (Shor's algorithm)

Part B: Security Threat and Mitigation

B.1 Threats to Diffie-Hellman on an unsecured network

Threat	Description	Mitigation
Man-in-the-Middle (MITM)	Attacker intercepts A and B and substitutes his own values, establishing separate keys with each user	Authenticate public values with digital signatures / certificates
Small-subgroup / invalid public value	Attacker sends a value of low order (e.g. 1 or p-1) forcing a predictable key	Validate that 1 < A < p-1 and A^q mod p = 1; use safe primes
Weak or small parameters (Logjam)	Small primes (512/1024-bit) or widely shared primes can be pre-computed and broken	Use at least 2048-bit groups (RFC 7919) or elliptic-curve DH
No forward secrecy	If a static private key is stolen later, all past sessions can be decrypted	Use fresh ephemeral keys (DHE / ECDHE) for every session

Threat	Description	Mitigation
Replay / denial of service	Old messages are replayed or the server is flooded with costly exponentiations	Nonces, timestamps, cookies / rate limiting

B.2 The most serious threat: Man-in-the-Middle attack

The network in the scenario is vulnerable to active attackers. Basic DH only guarantees that **someone** shares a key with you, not **who** that someone is. A MITM attacker (Mallory) who can intercept and modify traffic exploits this as follows:

1. Alice sends A = g^a mod p intended for Bob; Mallory intercepts it.
2. Mallory picks her own secret m and sends M = g^m mod p to Bob, pretending to be Alice.
3. Bob replies with B = g^b mod p; Mallory intercepts it and sends M to Alice, pretending to be Bob.
4. Alice computes K1 = M^a = g^(am) mod p (shared with Mallory); Bob computes K2 = M^b = g^(bm) mod p (shared with Mallory).
5. Mallory decrypts messages from Alice with K1, reads or alters them, re-encrypts them with K2 and forwards them to Bob (and vice versa). Neither user notices.

Figure 3 illustrates the attack. The red arrows are the forged public values injected by Mallory; the result is two different keys, K1 and K2, both known to the attacker.

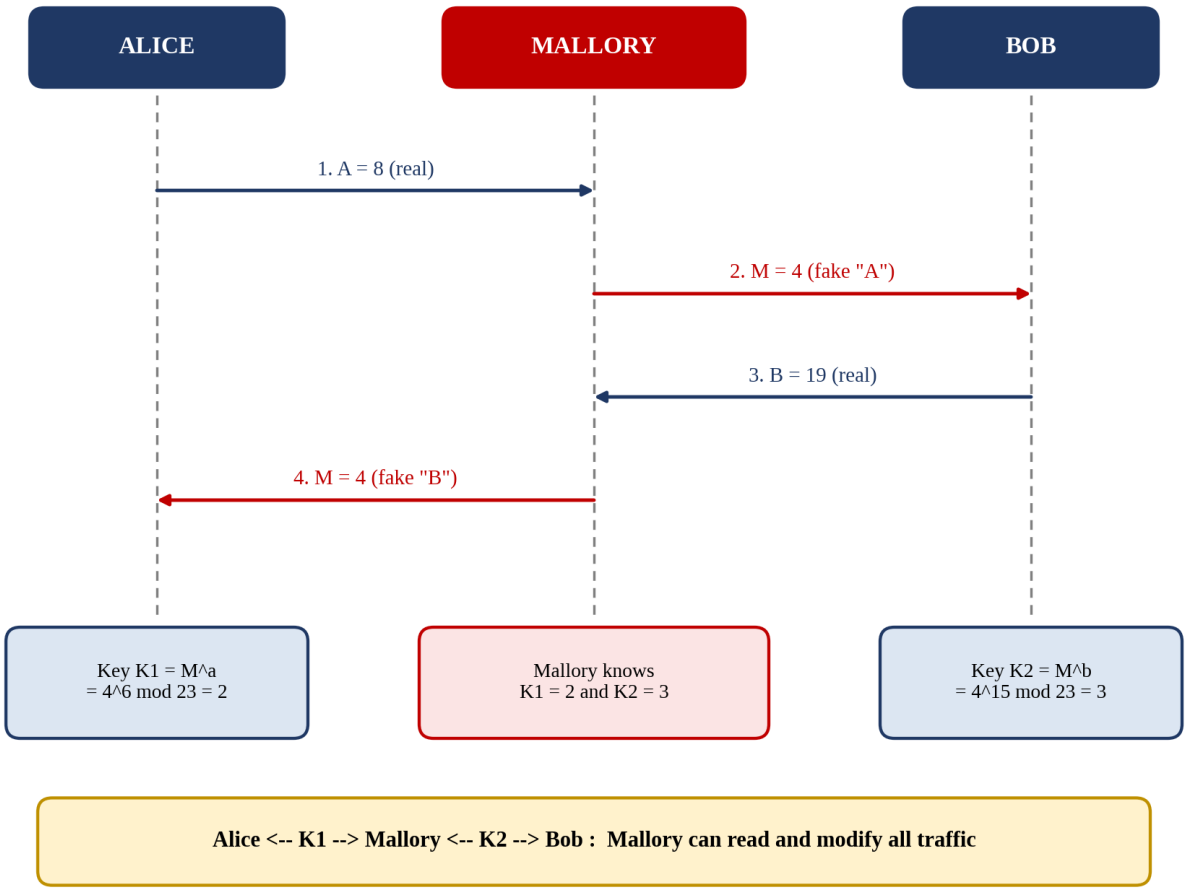


Figure 3: Man-in-the-Middle attack on unauthenticated Diffie-Hellman

Numerical illustration

Using p = 23, g = 5, a = 6, b = 15 and Mallory's secret m = 4 (so M = 5^4 mod 23 = 4):

Link	Calculation	Key obtained
Alice – Mallory	Alice: M^a = 4^6 mod 23; Mallory: A^m = 8^4 mod 23	K1 = 2
Bob – Mallory	Bob: M^b = 4^15 mod 23; Mallory: B^m = 19^4 mod 23	K2 = 3

Alice and Bob believe they share one key, but in reality Alice holds K1 = 2 and Bob holds K2 = 3, both known to Mallory. The attacker has full read/write control of the "secure" channel.

B.3 Solution: Authenticated Diffie-Hellman

The root cause is the absence of authentication. The standard remedy is to **bind each DH public value to the sender's identity** using digital signatures and certificates issued by a trusted Certificate Authority (CA). This is the basis of the **Station-to-Station (STS) protocol** and of the handshake in TLS 1.3 (ECDHE with signed handshake) and IPsec IKEv2.

1. Alice and Bob each hold a long-term signature key pair and an X.509 certificate issued by a trusted CA.

2. Alice generates an ephemeral value A = g^a mod p and sends A together with her certificate.
3. Bob generates B = g^b mod p, computes K = A^b mod p, and signs both public values: SigB = Sign(PrivB, B || A). He sends B, his certificate and SigB (optionally encrypted under K).
4. Alice verifies Bob's certificate against the trusted CA, verifies SigB with Bob's public key, and computes K = B^a mod p.
5. Alice sends SigA = Sign(PrivA, A || B); Bob verifies it in the same way.
6. Both derive session keys from K through a KDF and start encrypted communication with AES-GCM, optionally confirming with a MAC over the handshake transcript.

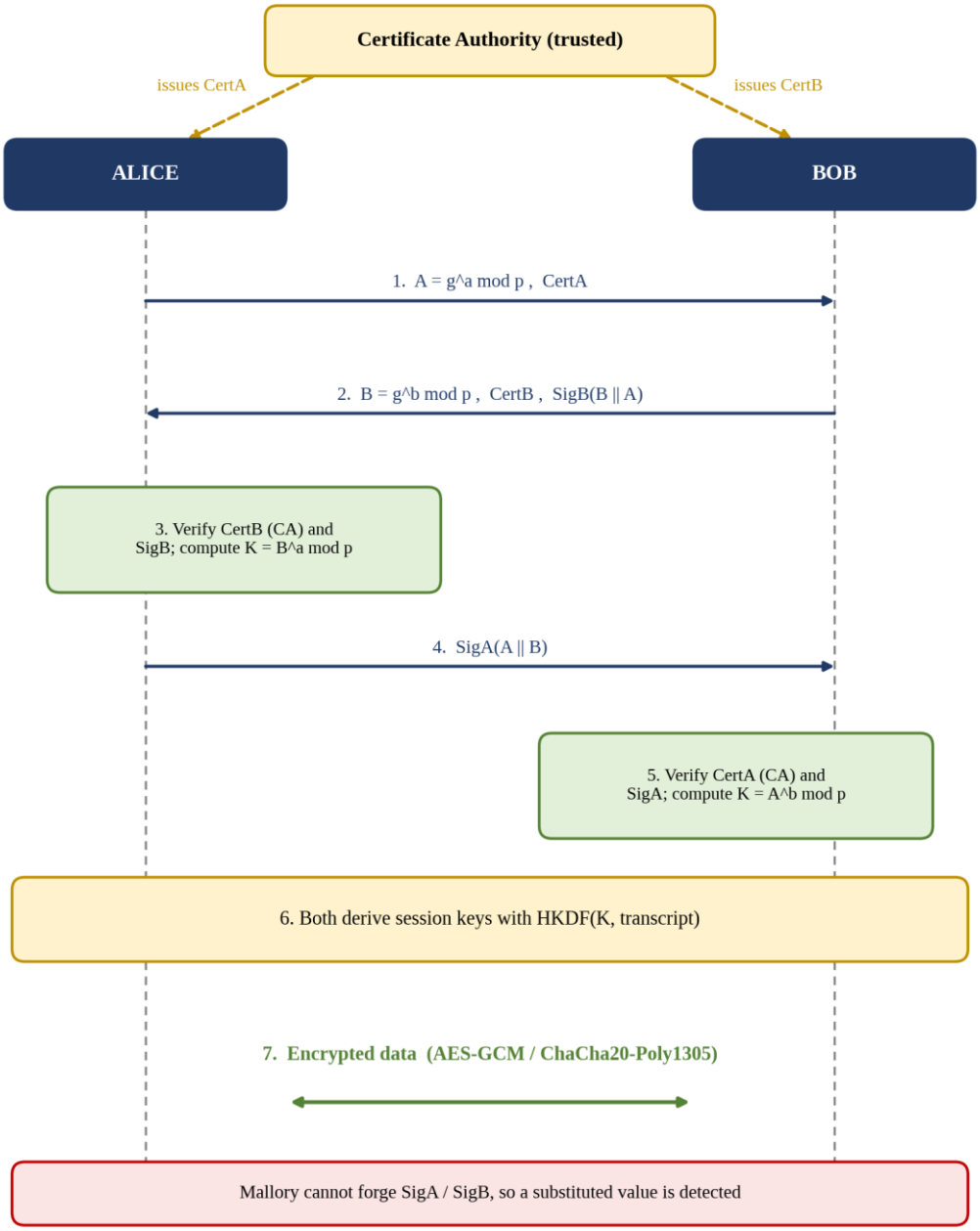


Figure 4: Authenticated Diffie-Hellman (Station-to-Station protocol) with certificates

If Mallory substitutes M for A or B, the signature check fails because she cannot produce a valid signature without the victim's private key, so the attack is detected and the session is aborted.

B.4 Supporting countermeasures

- **Strong parameters:** use a safe prime p of at least 2048 bits (standard groups from RFC 7919) or move to **ECDH** with Curve25519 / P-256, which offers equal strength with smaller keys.
- **Ephemeral keys (DHE/ECDHE):** generate new a and b for every session so that compromise of long-term keys never exposes past traffic (**perfect forward secrecy**).
- **Public-value validation:** reject values outside the range $1 < A < p-1$ and check subgroup membership to stop small-subgroup attacks.
- **Key derivation and key confirmation:** run K through HKDF and verify a MAC over the transcript to ensure both sides derived the same keys.
- **Replay and DoS protection:** include nonces/timestamps, use stateless cookies and rate limiting.
- **Pre-shared key or password authentication (alternative):** where certificates are impractical, authenticate the exchange with a PSK or a password-authenticated protocol such as SRP.

Before accepting any peer value, each side should run the checks shown in **Figure 5**. Failing any check immediately aborts the session, so a forged, malformed or replayed value never contributes to a key.

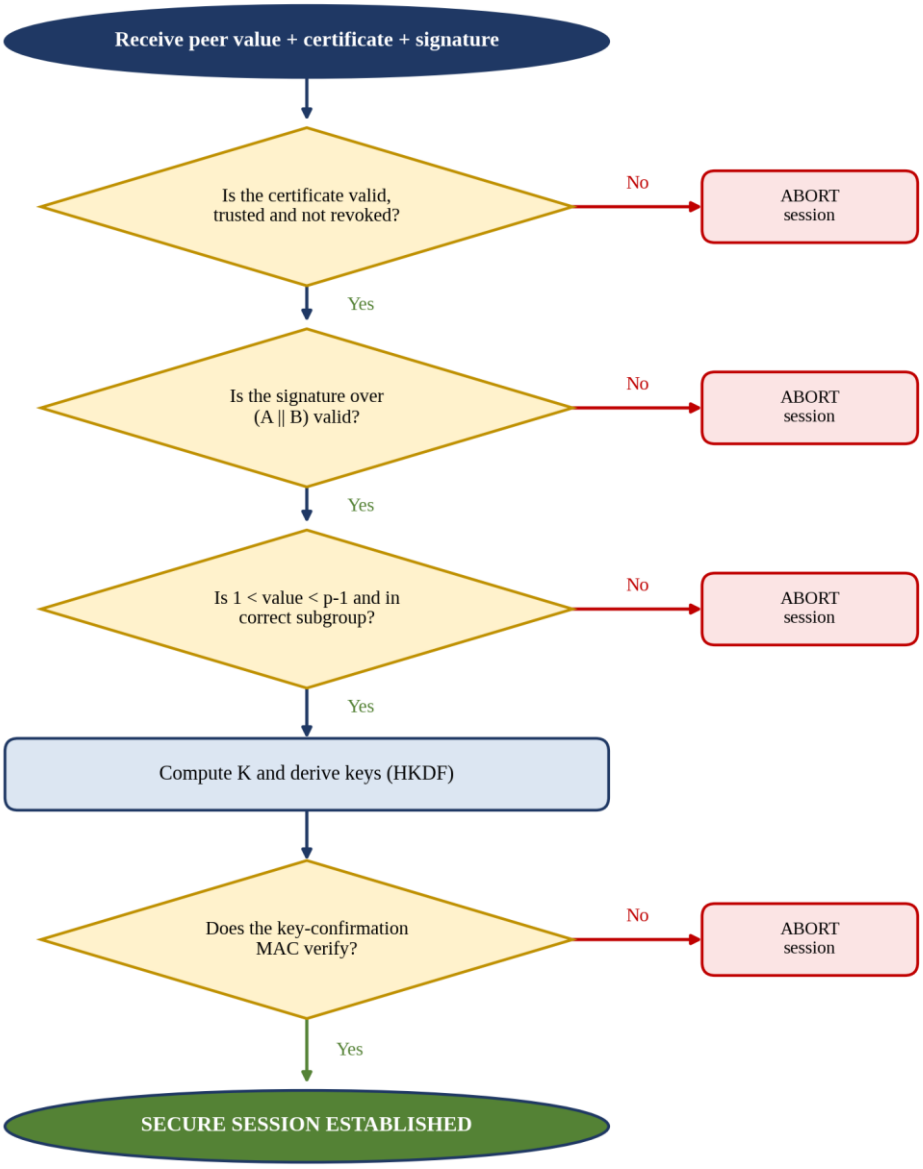


Figure 5: Validation flowchart for the received Diffie-Hellman handshake

B.5 Recommended design for the scenario

Component	Choice	Reason
Key agreement	ECDHE (Curve25519) or DHE with ≥ 2048-bit group	Strong, efficient, forward secrecy
Authentication	RSA-PSS / ECDSA signatures with CA-issued certificates	Defeats MITM by proving identity
Key derivation	HKDF-SHA256 over shared secret and transcript	Produces independent encryption and MAC keys
Data protection	AES-256-GCM or ChaCha20-Poly1305	Confidentiality and integrity of messages
Session management	New ephemeral keys per session, periodic re-keying	Limits damage from any key compromise

Question 4: Public Key Cryptography for Email Security

Scenario Understanding

An organization wants to secure its e-mail using **RSA**. E-mail travels through several untrusted servers in plain text by default, so it can be read (loss of **confidentiality**), altered (loss of **integrity**), or forged (loss of **authenticity** and **non-repudiation**). RSA, an asymmetric algorithm, provides a solution: each user has a **public key** (shared openly) and a **private key** (kept secret), so no prior secret has to be exchanged. The organization needs to understand (A) how RSA encrypts and decrypts messages and (B) how keys are distributed and managed, since the strength of the whole system depends on them.

Part A: RSA Encryption and Decryption for Email

A.1 Principle of RSA

RSA (Rivest–Shamir–Adleman, 1977) is based on the difficulty of **factoring** the product of two large primes. Multiplying primes p and q is easy, but recovering them from n = p x q is infeasible for a 2048-bit n. Encryption is performed with the public key (e, n), and decryption is possible only with the private key (d, n), which acts as a **trapdoor**.

A.2 Key generation (done once per user)

1. Select two large distinct random primes p and q (1024 bits each for a 2048-bit key).
2. Compute the modulus n = p x q.
3. Compute Euler's totient φ(n) = (p - 1)(q - 1).
4. Choose a public exponent e with 1 < e < φ(n) and gcd(e, φ(n)) = 1 (commonly e = 65537).
5. Compute the private exponent d = e^(-1) mod φ(n), i.e. e x d ≡ 1 (mod φ(n)), with the extended Euclidean algorithm.
6. Publish the **public key (e, n)**; keep the **private key (d, n)** (and p, q, φ(n)) secret, then destroy p, q and φ(n) or store them safely.

The key-generation procedure is summarised in **Figure 6**, with the values of the worked example shown alongside each step.

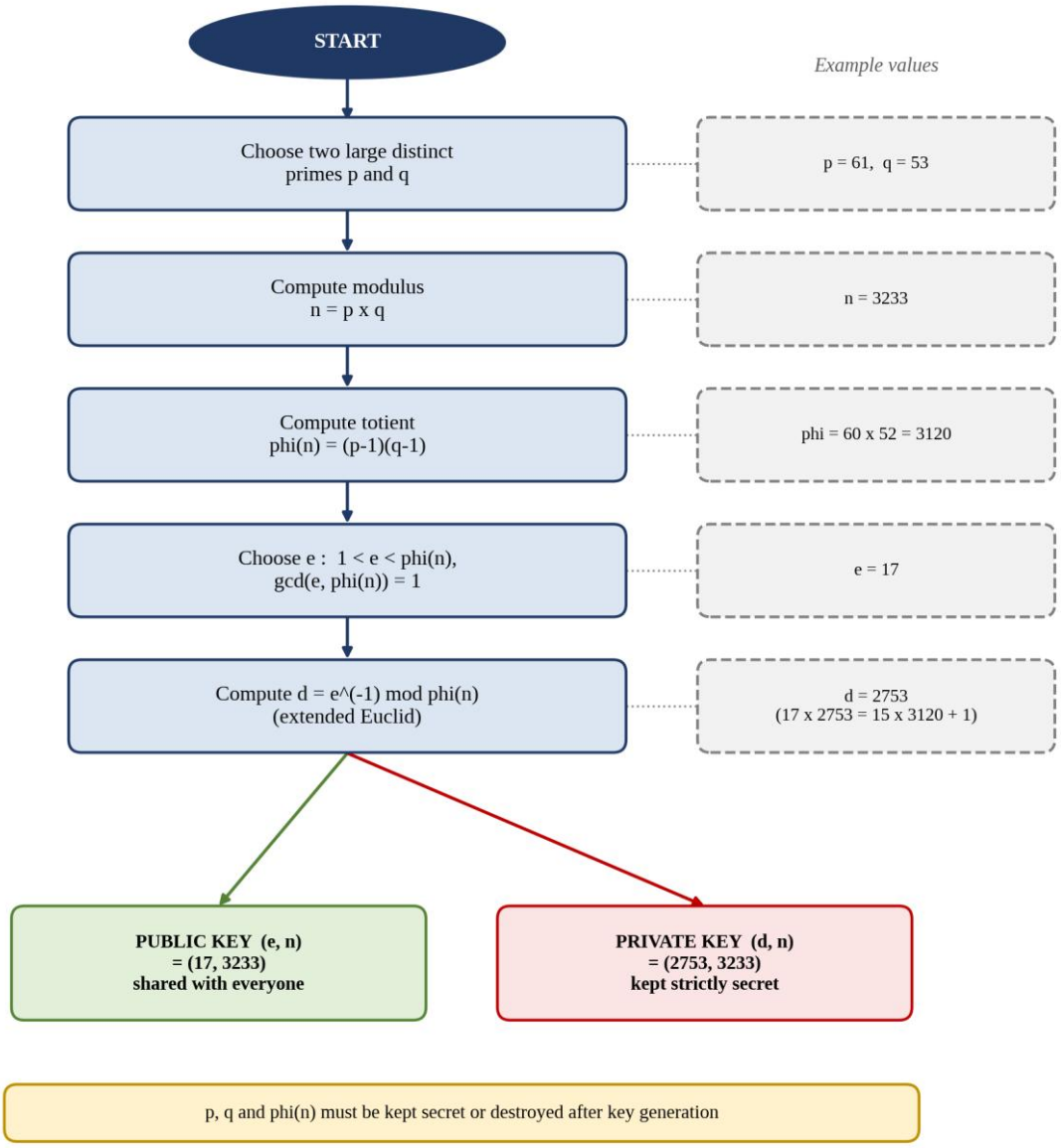


Figure 6: RSA key generation flowchart with example values

A.3 Encryption and decryption formulas

```
Encryption:  C = M^e mod n
Decryption:  M = C^d mod n
```

Decryption works because e x d = 1 + k·φ(n), and by Euler's theorem M^(1 + kφ(n)) ≡ M (mod n).

A.4 Worked numerical example

Step	Calculation	Result
Choose primes	p = 61, q = 53	—
Modulus	n = 61 x 53	n = 3233
Totient	φ(n) = 60 x 52	φ(n) = 3120
Public exponent	e = 17, gcd(17, 3120) = 1	Public key (17, 3233)

Step	Calculation	Result
Private exponent	d = 17^(-1) mod 3120; check 17 x 2753 = 46801 = 15 x 3120 + 1	Private key (2753, 3233)
Message	M = 65 (the letter "A")	M = 65
Encryption (by sender)	C = 65^17 mod 3233	C = 2790
Decryption (by receiver)	M = 2790^2753 mod 3233	M = 65

A.5 Using RSA in the e-mail scenario

Suppose Alice wants to send a confidential e-mail to Bob.

1. Alice obtains Bob's **authentic public key** (e_B, n_B) from a directory or from his certificate.
2. Alice writes the message M and encrypts it with Bob's public key: C = M^(e_B) mod n_B.
3. The ciphertext C is sent by e-mail. Anyone who intercepts it on the way (mail servers, attackers) sees only unreadable data.
4. Bob decrypts with his private key: M = C^(d_B) mod n_B. Only Bob holds d_B, so only Bob can read the message.

Figure 7 shows the flow of an encrypted e-mail from Alice to Bob. Only the public key of the receiver is needed by the sender, and only the receiver's private key can reverse the operation.

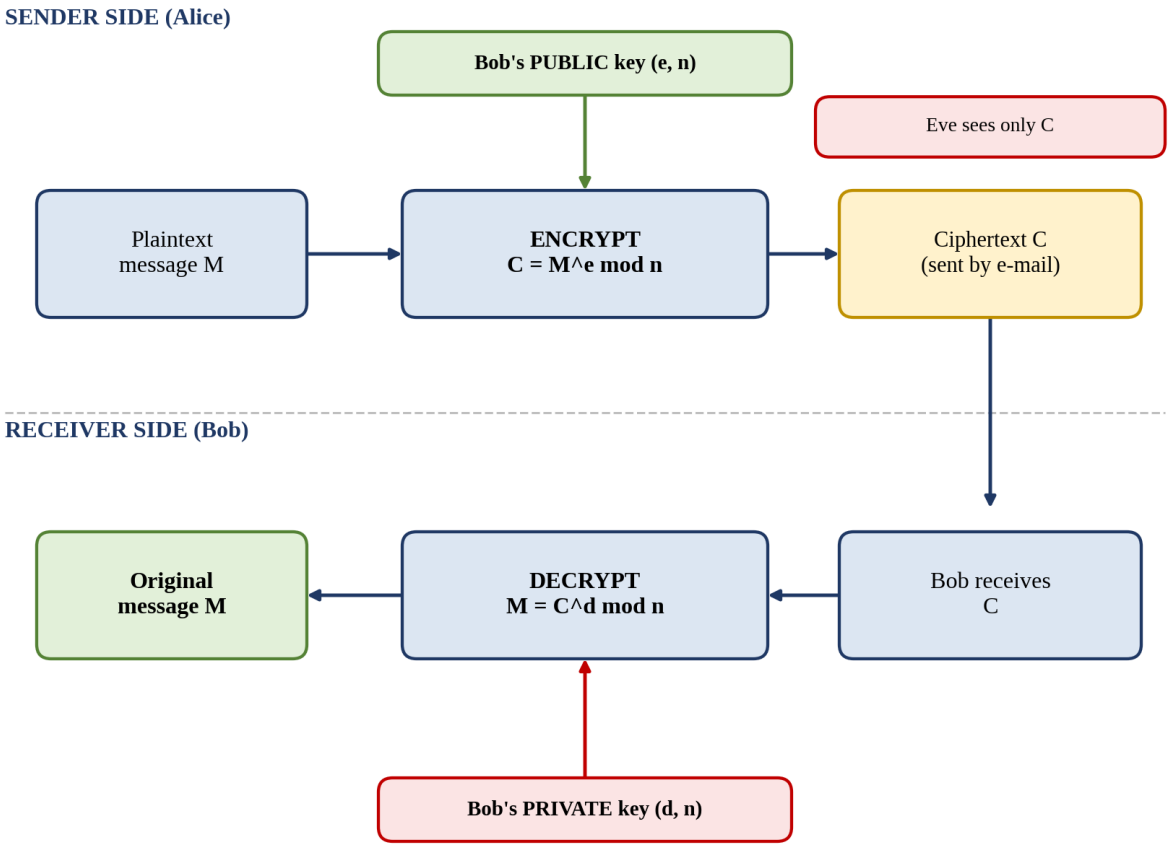


Figure 7: RSA encryption and decryption flow for e-mail

This provides **confidentiality**. For **authenticity and non-repudiation**, Alice additionally signs the message with her private key, and Bob verifies the signature with Alice's public key:

```
Signature:     S = H(M)^(d_A) mod n_A
Verification:  H(M) =? S^(e_A) mod n_A
```

In the example above, signing the value 65 with d = 2753 gives S = 65^2753 mod 3233 = 588, and the verifier computes 588^17 mod 3233 = 65, which matches the message, proving that it came from the owner of the private key. In practice the signature is applied to a hash of the e-mail (SHA-256).

The signing and verification process is shown in **Figure 8**. If even one bit of the e-mail changes in transit, the hash computed by the receiver differs from the hash recovered from the signature and the message is rejected.

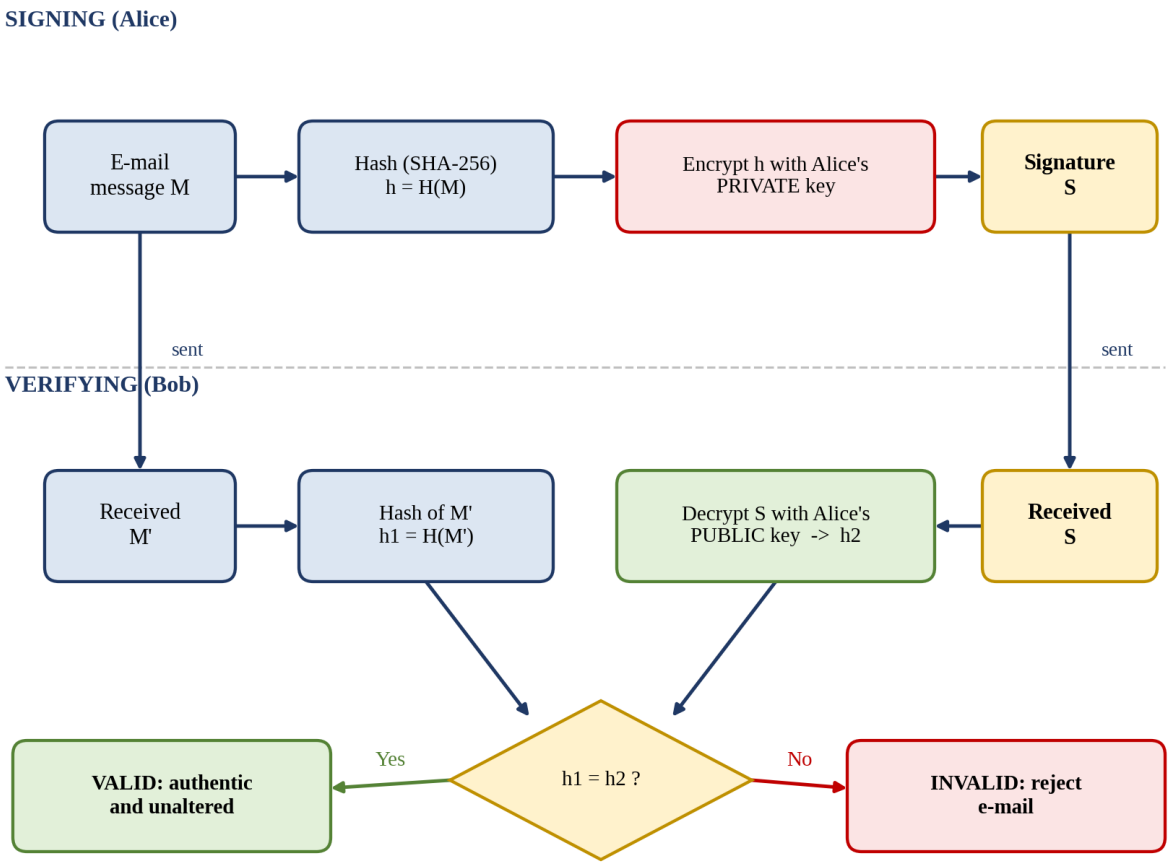


Figure 8: RSA digital signature – signing and verification

Security goal	RSA operation	Key used
Confidentiality	Sender encrypts; receiver decrypts	Receiver's public key to encrypt; receiver's private key to decrypt
Authentication / integrity / non-repudiation	Sender signs hash; receiver verifies	Sender's private key to sign; sender's public key to verify
Both (sign, then encrypt)	Sign with own private key, encrypt with recipient's public key	Both pairs used

A.6 Practical implementation: hybrid encryption

RSA is roughly 1000 times slower than AES and can only encrypt data smaller than the modulus (about 190 bytes for RSA-2048 with OAEP), so real e-mail systems such as **S/MIME** and **PGP/OpenPGP** use a **hybrid approach**:

1. The sender generates a random one-time **AES session key** Ks.
2. The e-mail body and attachments are encrypted with Ks (AES-256-GCM).
3. Ks is encrypted with the recipient's RSA public key using **RSA-OAEP** padding.
4. The sender signs the hash of the message with RSA-PSS (or PKCS#1 v1.5) and attaches the signature.

5. The recipient decrypts Ks with the private key, decrypts the body with Ks, and verifies the signature.

The complete hybrid workflow, as used by S/MIME, is shown in **Figure 9**.

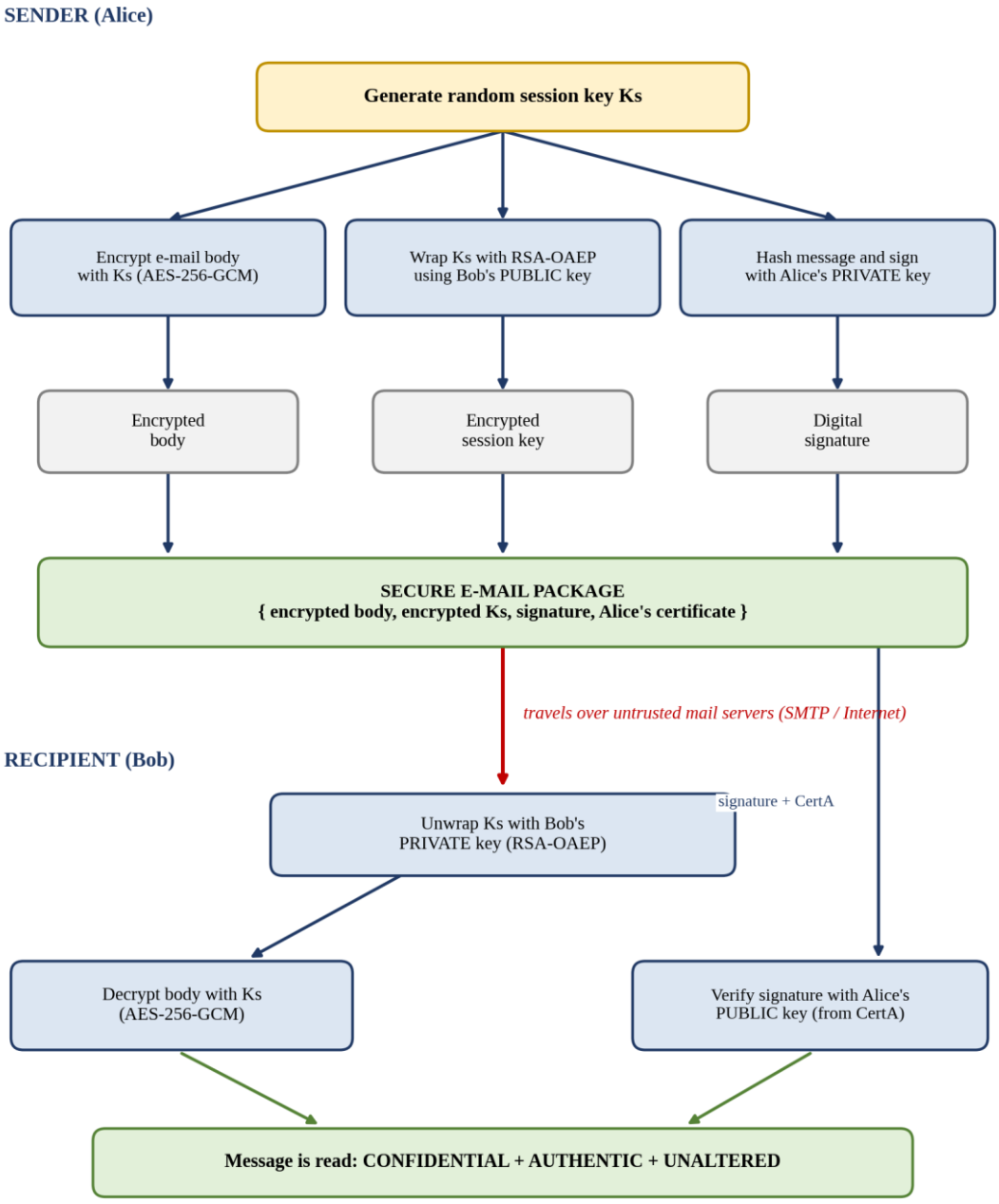


Figure 9: Hybrid secure e-mail: AES for the body, RSA for the key and the signature

The same message can be sent to many recipients by wrapping Ks separately under each recipient's public key, which makes group e-mail efficient.

A.7 Security considerations for RSA

- **Key size:** at least 2048 bits (about 112-bit security); 3072 bits for long-term protection (NIST SP 800-57).
- **Padding:** never use "textbook" RSA. It is deterministic and malleable; use OAEP for encryption and PSS for signatures.

- **Random primes:** weak random-number generators lead to guessable or shared primes and factorable moduli.
- **Side-channel and padding-oracle attacks:** use constant-time libraries and do not reveal decryption errors.
- **Quantum threat:** Shor's algorithm could break RSA; plan migration to post-quantum algorithms for long-term secrets.

Part B: Role of Key Distribution and Key Management

B.1 Why key distribution matters

RSA removes the need to share a secret key, but it introduces a different requirement: **the sender must be sure that a public key really belongs to the intended recipient**. If an attacker can replace Bob's public key with his own (a **public-key substitution** or MITM attack), Alice will unknowingly encrypt messages that the attacker can read, and the mathematical strength of RSA becomes irrelevant. Public keys need **authenticity and integrity**, though not secrecy, while private keys need **strict secrecy**.

B.2 Methods of public-key distribution

Method	How it works	Strengths / weaknesses
Public announcement	Users broadcast their key by e-mail or website	Simple, but anyone can forge an announcement
Public directory	A trusted directory stores name/key pairs	Easy lookup; the directory is a single point of attack
Public-key authority	An online authority supplies signed keys on request	Stronger control, but a bottleneck and always online
Public-key certificates (PKI / X.509)	A Certificate Authority signs a certificate binding the identity to the public key	Scalable and offline verification; relies on trust in the CA (used by S/MIME)
Web of trust (PGP)	Users sign one another's keys; trust is transitive	No central authority, but harder to manage in large organizations

Figure 10 shows a typical certificate hierarchy: the Root CA signs the Issuing CA, which signs employee certificates.

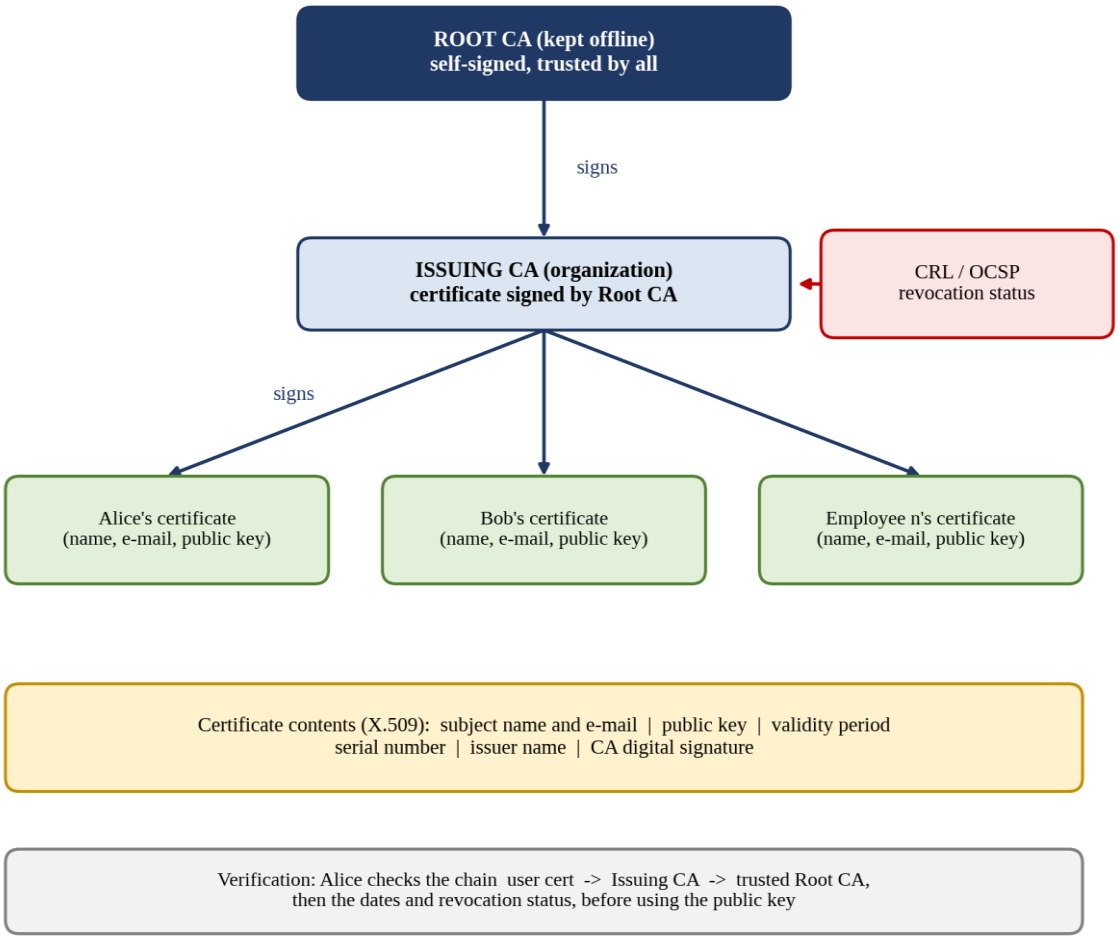


Figure 10: Public-key infrastructure (PKI) certificate hierarchy

For an organization, the best choice is **PKI with an internal (or trusted external) Certificate Authority**. Each employee receives an X.509 certificate containing the user's name and e-mail address, public key, validity period, serial number, issuing CA and the CA's digital signature. A mail client checks that the certificate chain leads to a trusted root, that it has not expired and that it has not been revoked before using the key.

B.3 Key management life cycle

Phase	Required action in the organization
Generation	Generate keys on the user's device or inside an HSM/smart card with a cryptographically secure random-number generator; at least 2048-bit RSA with e = 65537.
Registration and certification	Verify the user's identity, then have the CA issue a certificate for the public key.
Distribution	Publish certificates in an LDAP directory or attach them to signed e-mails; never distribute private keys.
Storage and protection	Store private keys encrypted (passphrase) or on hardware tokens/HSMs; restrict access and keep access logs.

Phase	Required action in the organization
Usage control	Use separate key pairs for signing and for encryption so that each key has a single purpose.
Rotation and renewal	Certificates expire (for example after 1–2 years); new keys are issued before expiry to limit exposure.
Revocation	If a key is lost or compromised or an employee leaves, revoke the certificate and publish it through a CRL or OCSP responder.
Backup, escrow and recovery	Keep encryption keys in secure escrow so old mail can be recovered; do not escrow signing keys, to maintain non-repudiation.
Archival and destruction	Keep old decryption keys for archived mail as needed; securely erase keys that are no longer required.

The life cycle in the table above is also shown graphically in **Figure 11**; the cycle repeats every time a key is renewed or replaced.

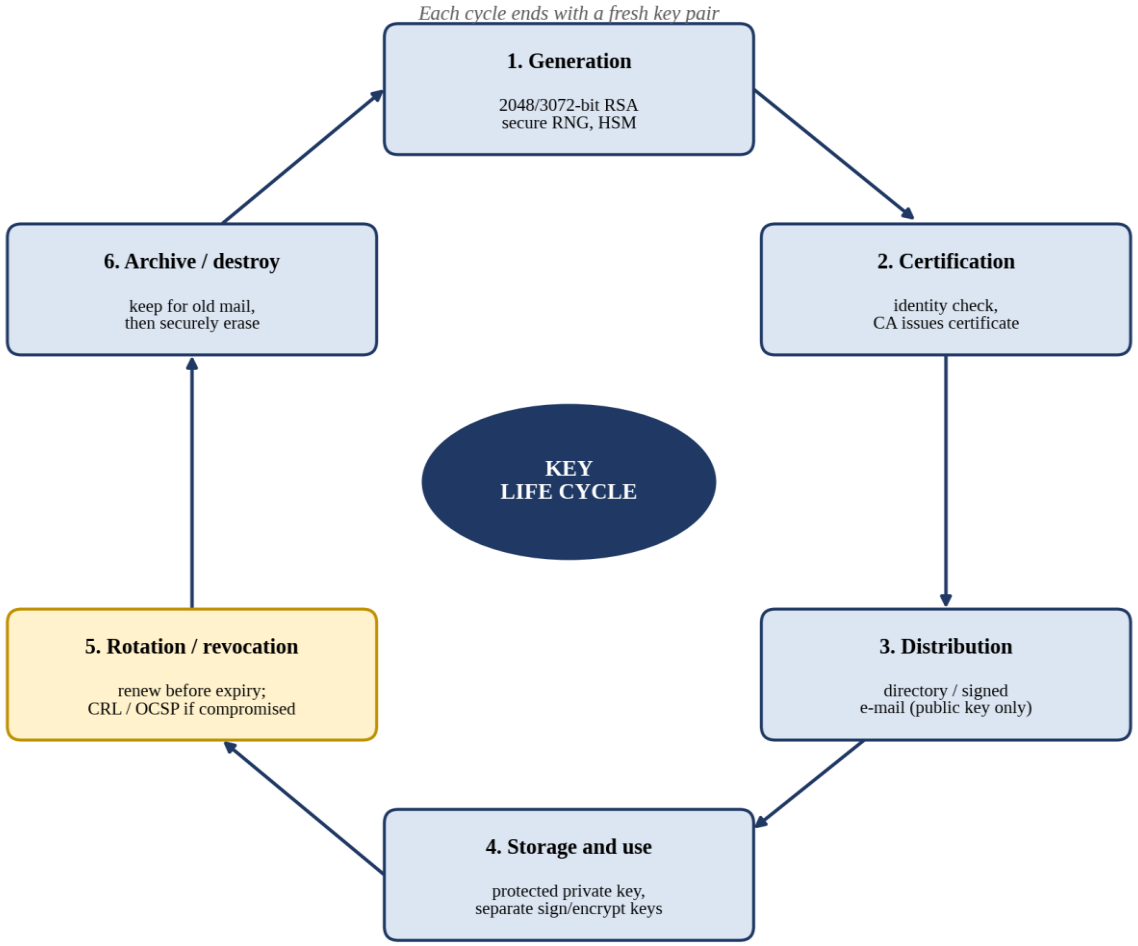


Figure 11: Key management life cycle

B.4 Threats from poor key management and countermeasures

Threat	Impact	Countermeasure
Public-key substitution / fake certificate	Attacker reads encrypted mail	CA-signed certificates, chain validation, fingerprint verification
Private-key theft	Attacker decrypts mail and forges signatures	HSM/smart card, strong passphrase, access control, immediate revocation
Expired or revoked key still in use	Compromised keys continue to be trusted	CRL/OCSP checks, automated renewal reminders
Weak random number generation	Predictable or shared primes can be factored	Certified RNG, key-generation audits
Key reuse for many purposes	Single compromise causes large damage	Separate encryption and signing keys, scheduled rotation
Lost private key	Old encrypted e-mail becomes unreadable	Secure key backup / escrow for encryption keys

B.5 Recommended policy for the organization

- Deploy an internal PKI (root CA kept offline, issuing CA online) and integrate it with the e-mail clients through S/MIME.
- Use RSA-3072 or RSA-2048 with SHA-256, OAEP and PSS, and AES-256-GCM for message bodies.
- Require hardware-protected private keys for senior staff and administrators.
- Maintain a formal key-management policy covering issuance, renewal, revocation, backup and audit, and train users to verify certificates and to report lost keys at once.
- Monitor developments in post-quantum cryptography and plan a migration path.

Conclusion for Question 4

RSA gives the organization both confidentiality (encrypt with the recipient's public key) and authenticity (sign with the sender's private key), and in practice it is combined with symmetric AES through hybrid encryption. However, RSA is only as secure as the way its keys are handled: public keys must be distributed authentically through a PKI, and private keys must be generated, stored, rotated and revoked under a disciplined key-management policy. Strong mathematics together with sound key management is what actually keeps e-mail secure.